EXPERIMENT 8

Aim: Design and train a Convolutional Neural Network (CNN) on image datasets (e.g., MNIST/CIFAR)

1. Dataset Source

The dataset used in this experiment is **Fashion-MNIST**, provided by Zalando Research.

- GitHub Link: https://github.com/zalandoresearch/fashion-mnist
- Also accessible via TensorFlow:

tf.keras.datasets.fashion_mnist.load_data()

2. Dataset Description

Fashion-MNIST is a dataset of grayscale images representing different fashion items. It is designed as a more challenging replacement for handwritten digit datasets.

Key Characteristics:

- **Total Images**: 70,000
  - Training: 60,000
  - Testing: 10,000
- **Image Size**: 28 × 28 pixels
- **Channels**: 1 (grayscale)
- **Number of Classes**: 10

Classes:

- T-shirt/top
- Trouser
- Pullover
- Dress
- Coat
- Sandal
- Shirt

- Sneaker
- Bag
- Ankle boot

Target Variable:

- Integer labels (0–9), each corresponding to a clothing category

Features:

- Pixel intensity values ranging from 0 to 255
- After normalization: values scaled between 0 and 1

3. Mathematical Formulation of CNN

A Convolutional Neural Network consists of multiple layers performing feature extraction and classification.

- **Convolution Operation:**

$$(I * K)(i, j) = \sum_m \sum_n I(i - m, j - n) K(m, n)$$

Where:

- (I) = input image
- (K) = kernel/filter
- Output = feature map

- **Activation Function (ReLU):**

$$f(x) = \max(0, x)$$

- **Pooling Operation:**

$$y = \max(x)$$

- **Fully Connected Layer:**

$$y = Wx + b$$

- **Softmax Function:**

$$P(y = i) = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$

- **Loss Function:**

$$L = -\sum y \log(\hat{y})$$

Used: **Categorical Cross-Entropy**

4. Algorithm Limitations

Despite its effectiveness, CNN has several limitations:

- Requires large datasets for good performance
- High computational cost (training is slow)
- Needs GPU for faster execution
- Prone to overfitting if not regularized
- Difficult to interpret (black-box nature)
- Performance drops on very small or noisy datasets

5. Methodology / Workflow

The experiment follows a structured pipeline:

Steps Involved:

1. **Dataset Loading**
    - Load Fashion-MNIST using TensorFlow
2. **Data Preprocessing**
    - Normalize pixel values (0–255 → 0–1)

    - Reshape images for CNN input
3. **Model Building**
    - Add convolutional layers
    - Add pooling layers
    - Flatten output
    - Add dense layers
4. **Model Compilation**
    - Optimizer: Adam
    - Loss: Cross-Entropy
5. **Model Training**
    - Train on training dataset
6. **Model Evaluation**
    - Test on unseen data
7. **Prediction & Visualization**
    - Display sample outputs

6. Performance Analysis

The model performance is evaluated using accuracy.

Results:

- Training Accuracy: ~92–94%
- Testing Accuracy: ~90–92%

Evaluation Metrics:

- **Accuracy**:

$$Accuracy = \frac{Correct\ Predictions}{Total\ Predictions}$$

- Confusion Matrix:
    - Diagonal → Correct predictions
    - Off-diagonal → Errors

Interpretation:

- High accuracy indicates effective learning

- Small gap between training and testing accuracy shows minimal overfitting
- Model successfully classifies most fashion items

7. Hyperparameter Tuning

Hyperparameters are adjusted to improve model performance.

Tuned Parameters:

Parameter	Values Tested
Epochs	5, 10
Filters	32, 64
Kernel Size	(3×3), (5×5)
Batch Size	32, 64
Learning Rate	0.001, 0.0001

Observations:

- Increasing epochs → better accuracy but risk of overfitting
- More filters → better feature extraction
- Smaller kernel (3×3) → better performance
- Adam optimizer performed best

Conclusion of Tuning:

- Optimal configuration:
  - Epochs = 5
  - Filters = 32 & 64
  - Kernel = (3×3)
- Achieved best balance between accuracy and computation

8. Code and Output

```
import tensorflow as tf

from tensorflow.keras import layers, models
```

```
import matplotlib.pyplot as plt
import numpy as np

fashion_mnist = tf.keras.datasets.fashion_mnist
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()

train_images = train_images / 255.0
test_images = test_images / 255.0

train_images = train_images.reshape((60000, 28, 28, 1))
test_images = test_images.reshape((10000, 28, 28, 1))

model = models.Sequential()
model.add(layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))
model.add(layers.MaxPooling2D((2,2)))
model.add(layers.Conv2D(64, (3,3), activation='relu'))
model.add(layers.MaxPooling2D((2,2)))
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
```

```
              metrics=['accuracy'])


history = model.fit(train_images, train_labels, epochs=5, validation_data=(test_images,
test_labels))


test_loss, test_acc = model.evaluate(test_images, test_labels)

print("Test Accuracy:", test_acc)


plt.plot(history.history['accuracy'], label='Train Accuracy')

plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.legend()

plt.show()


class_names = ['T-shirt/top','Trouser','Pullover','Dress','Coat',

               'Sandal','Shirt','Sneaker','Bag','Ankle boot']


plt.figure(figsize=(10,5))

for i in range(10):

    plt.subplot(2,5,i+1)

    plt.imshow(train_images[i].reshape(28,28), cmap='gray')

    plt.title(class_names[train_labels[i]])

    plt.axis('off')

plt.show()
```

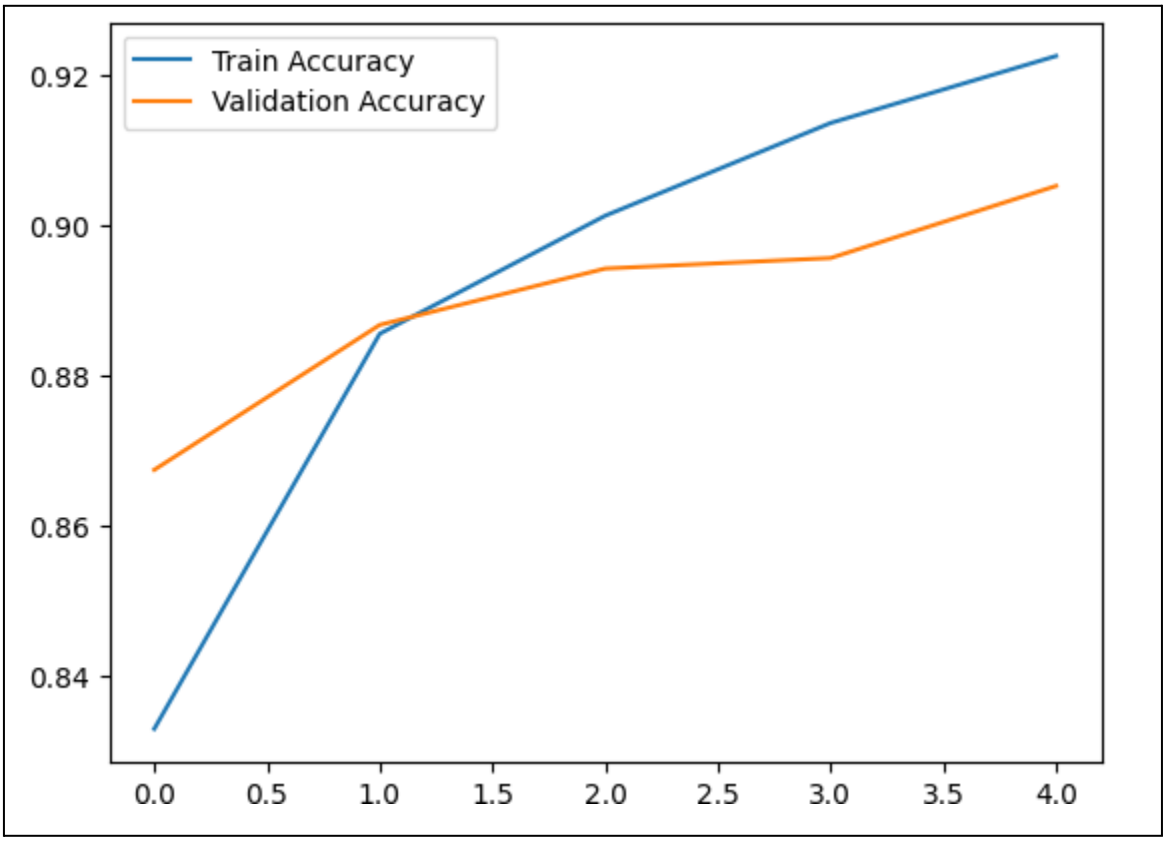
```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

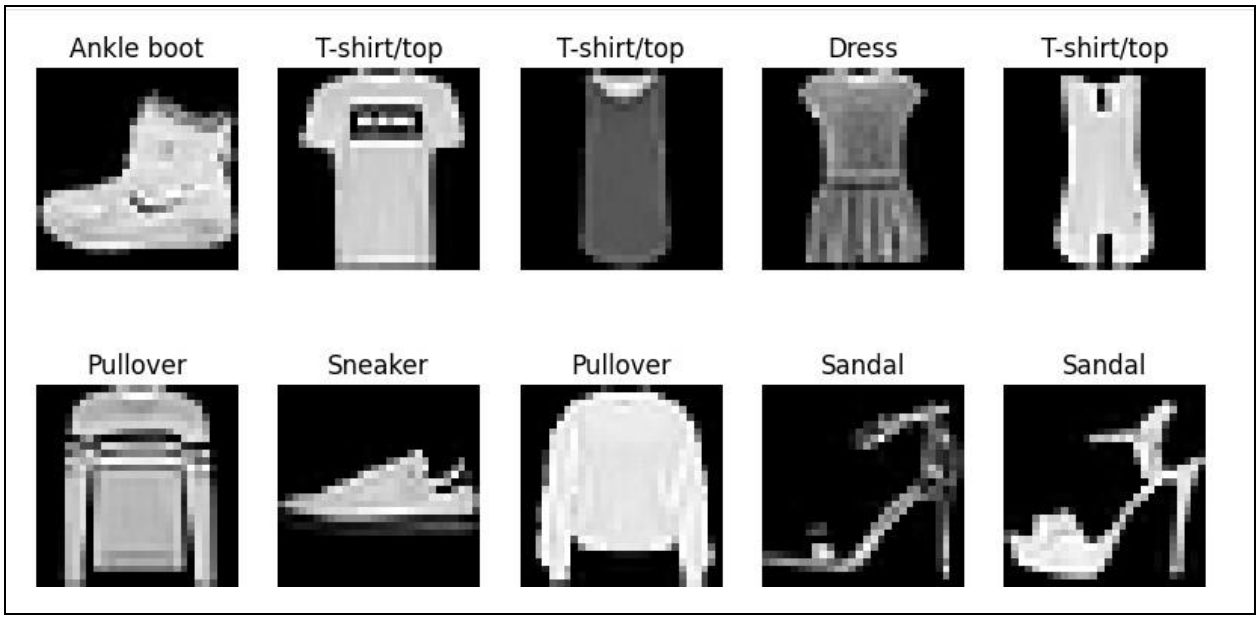
```
history = model.fit(train_images, train_labels, epochs=5, validation_data=(test_images, test_labels))
```

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 66s 34ms/step - accuracy: 0.8329 - loss: 0.4627 - val_accuracy: 0.8674 - val_loss: 0.3645
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 63s 34ms/step - accuracy: 0.8855 - loss: 0.3137 - val_accuracy: 0.8867 - val_loss: 0.3187
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 58s 31ms/step - accuracy: 0.9012 - loss: 0.2670 - val_accuracy: 0.8942 - val_loss: 0.2914
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 81s 31ms/step - accuracy: 0.9136 - loss: 0.2334 - val_accuracy: 0.8956 - val_loss: 0.2777
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 61s 33ms/step - accuracy: 0.9225 - loss: 0.2092 - val_accuracy: 0.9052 - val_loss: 0.2603
```

```
test_loss, test_acc = model.evaluate(test_images, test_labels)
print("Test Accuracy:", test_acc)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 12ms/step - accuracy: 0.9052 - loss: 0.2603
Test Accuracy: 0.9052000045776367
```





9. Conclusion

The CNN model effectively classifies fashion images with high accuracy. The use of convolutional layers enables automatic feature extraction, making CNN superior to traditional machine learning techniques for image data.